

Computer Architecture: *Memory Types*

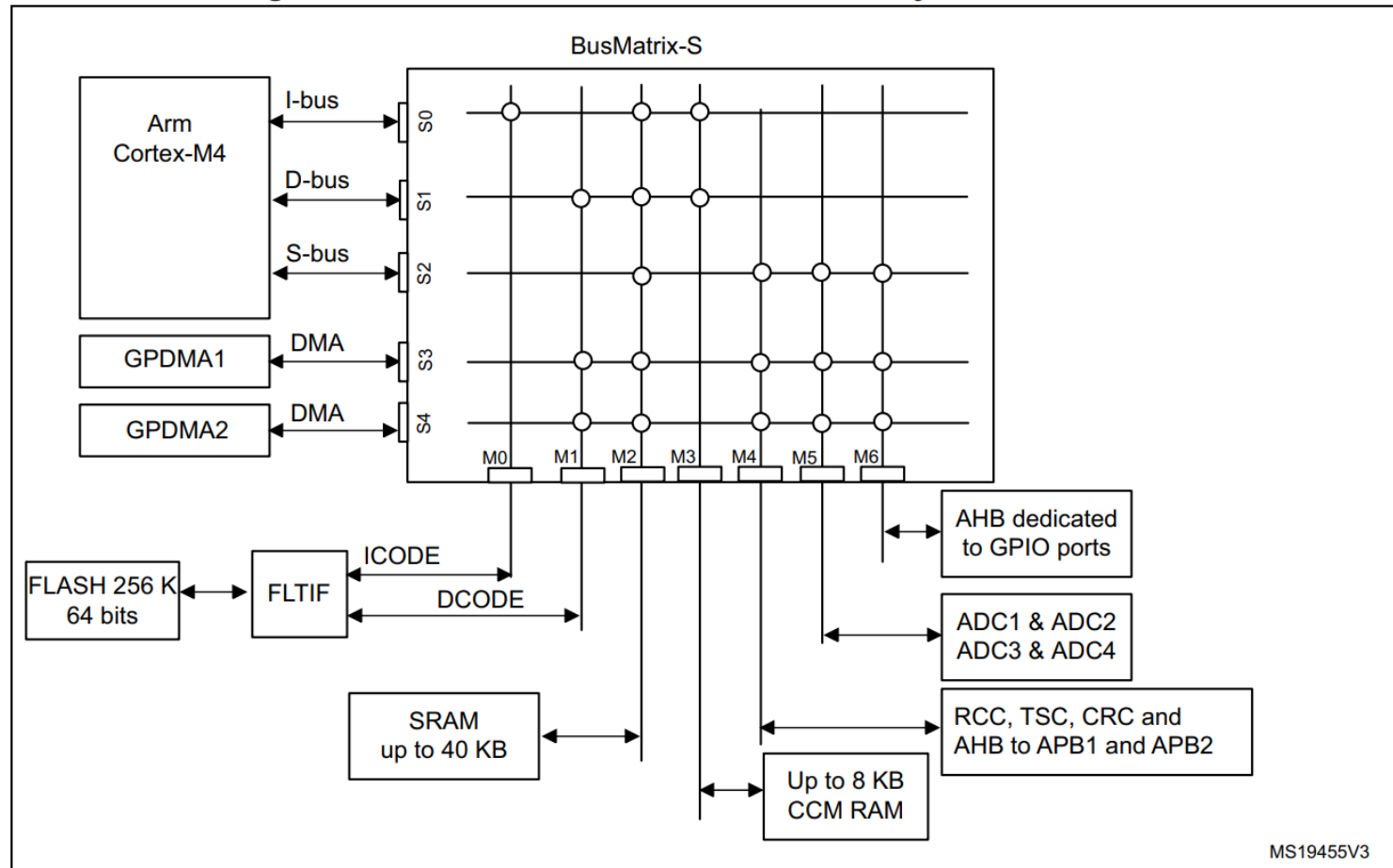
Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How does the machine run? (Control Flow)
- How does the machine store data?
- How does the machine interface using buses?
- What does it mean to have “allowed” memory?
- **Why is some memory faster & slower?**
- How does the machine speed up data access?

Why would a bus not respond on time?

Figure 1. STM32F303xB/C and STM32F358xC system architecture



- Prior lectures looked at busses
- This lecture will look in detail at why we have many choices.

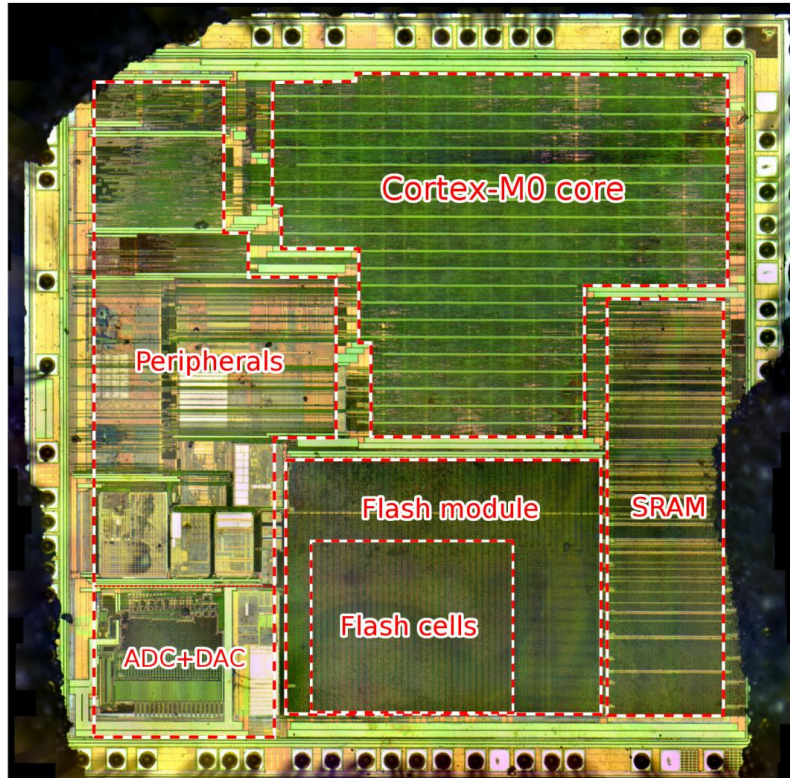
Properties of Memory

- Speed
- Density
- Cost
- Volatility

Comparison of Memory Types

	Speed	Density	Cost	Volatility
Static RAM	High	Low	High	Requires Power
Dynamic RAM	Very High	High	Medium	Requires
Flash	Slower	Very High	Low	>10 year without power

Typical Small Microcontroller

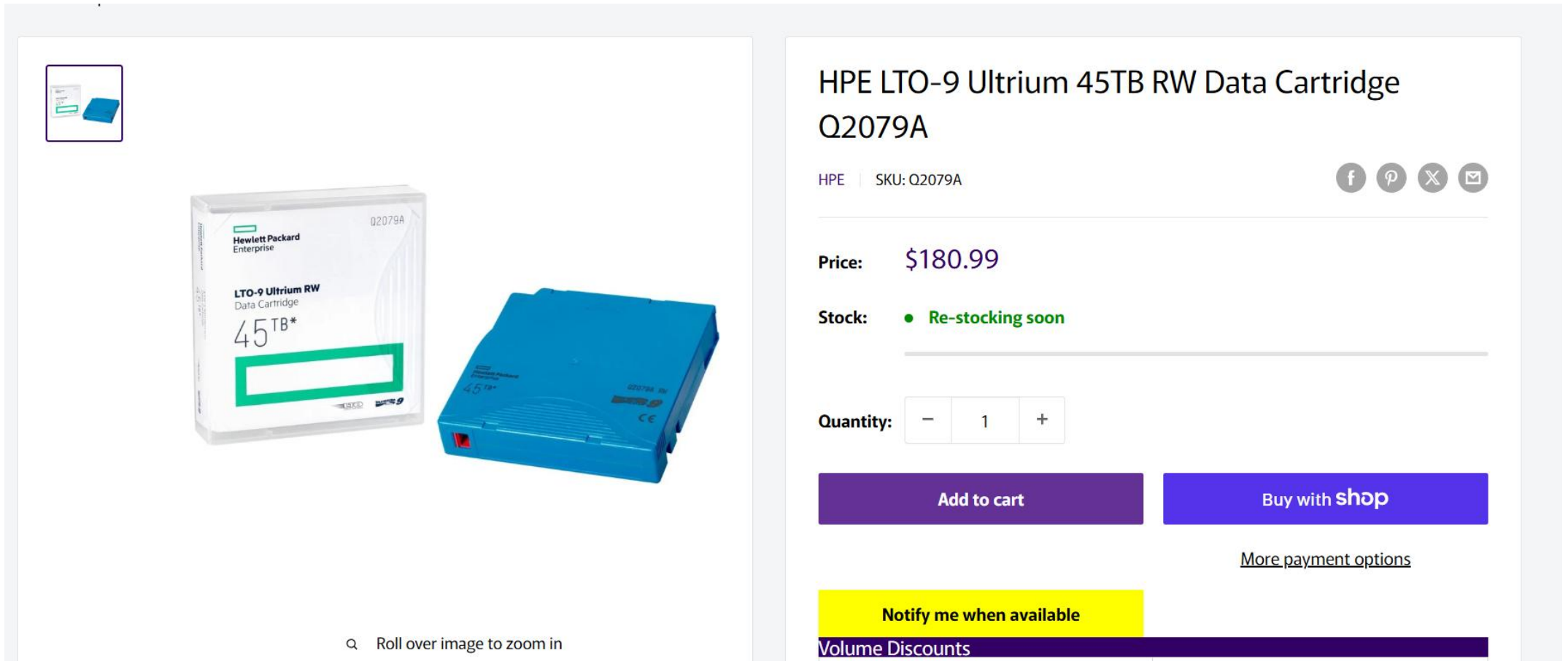


STM32F051R8T6:

- Cortex-M0 (simpler than M4 we are using)
- 64 Kbyte Flash
- 8 Kbyte SRAM

<https://www.usenix.org/system/files/conference/woot17/woot17-paper-obermaier.pdf>

Why Tapes are Still used in 2026



HPE LTO-9 Ultrium 45TB RW Data Cartridge Q2079A

HPE | SKU: Q2079A

Price: \$180.99

Stock: ● Re-stocking soon

Quantity: - 1 +

Add to cart Buy with shop

[More payment options](#)

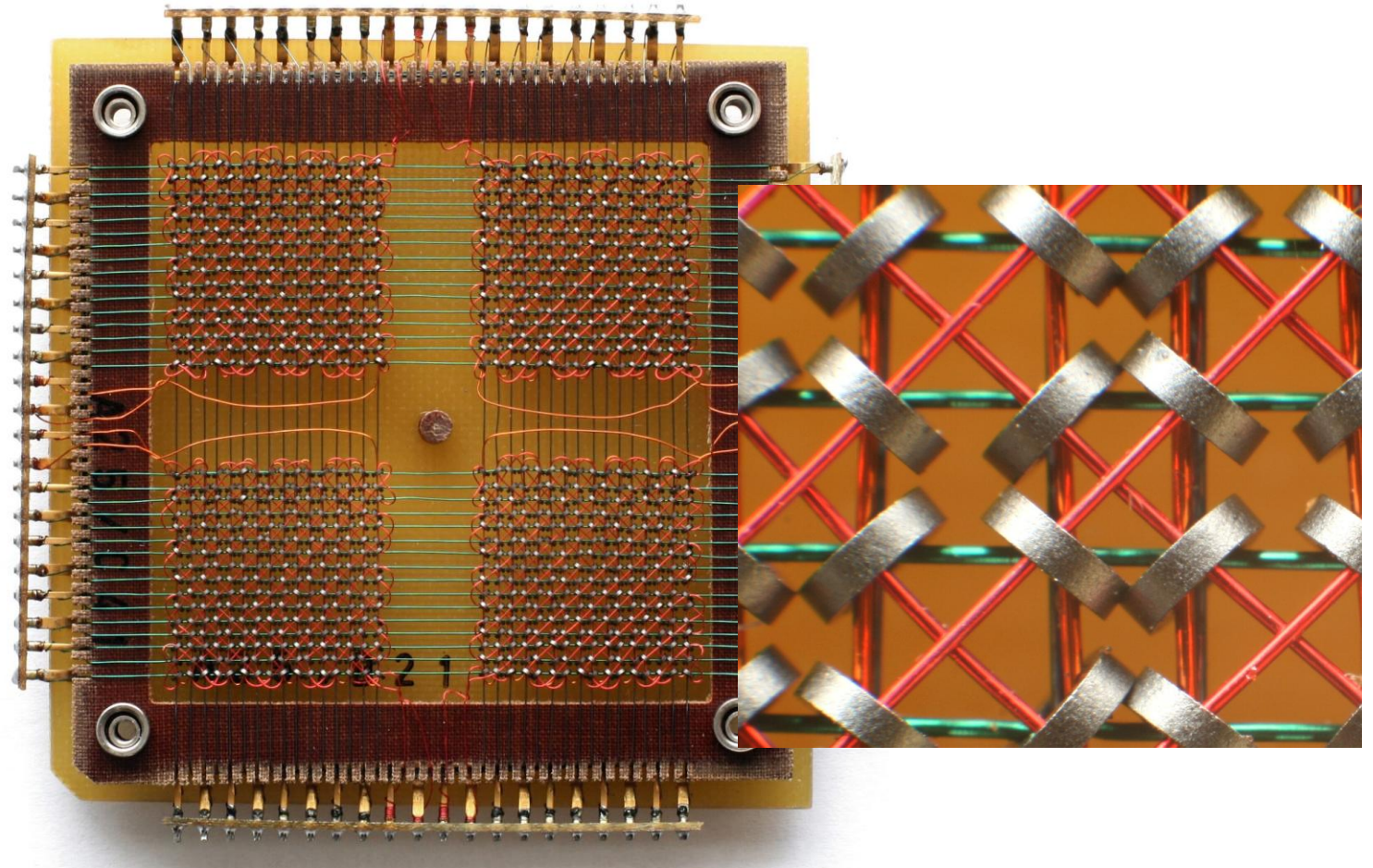
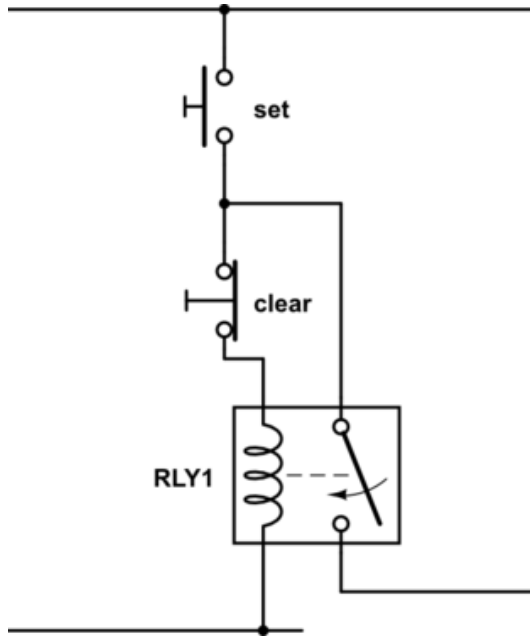
Notify me when available

Volume Discounts

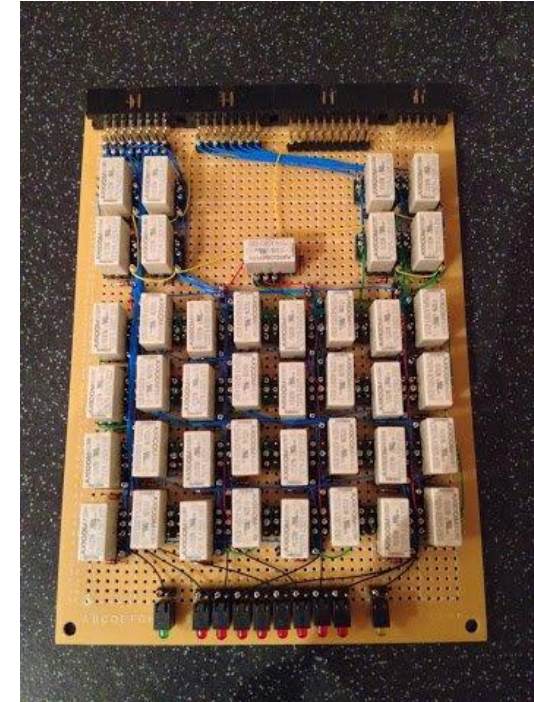
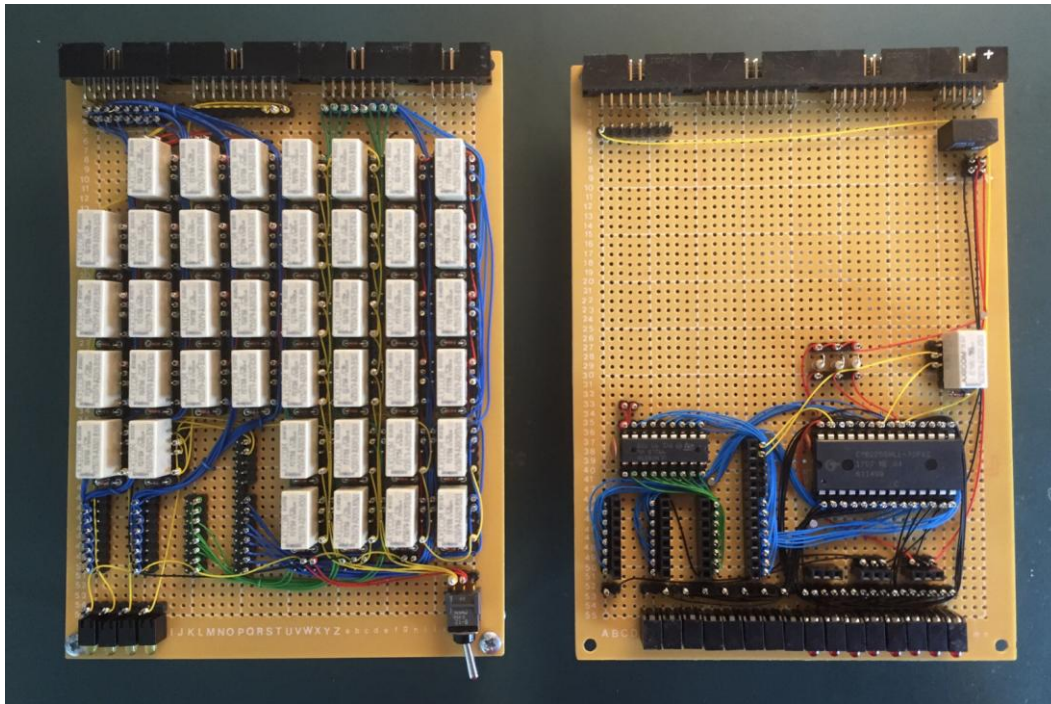
Roll over image to zoom in

Speed: **VERY SLOW** Density: **VERY HIGH** Cost: **VERY LOW** Volatility: **VERY LONG**

History of Computer Memory



Modern Relay Computers



Modern SRAM chips used in combination with MANY relays..

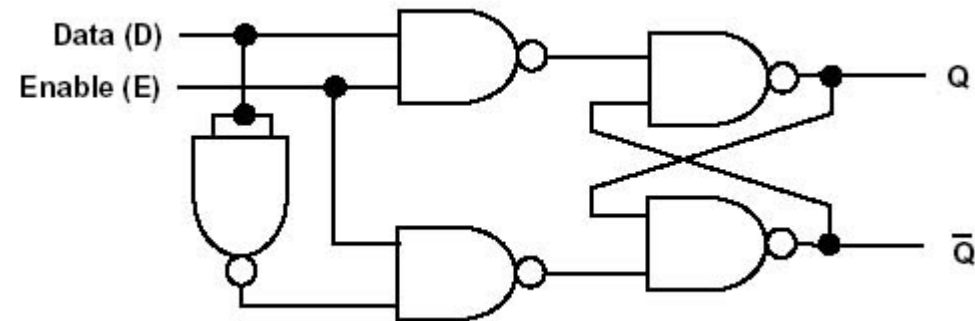
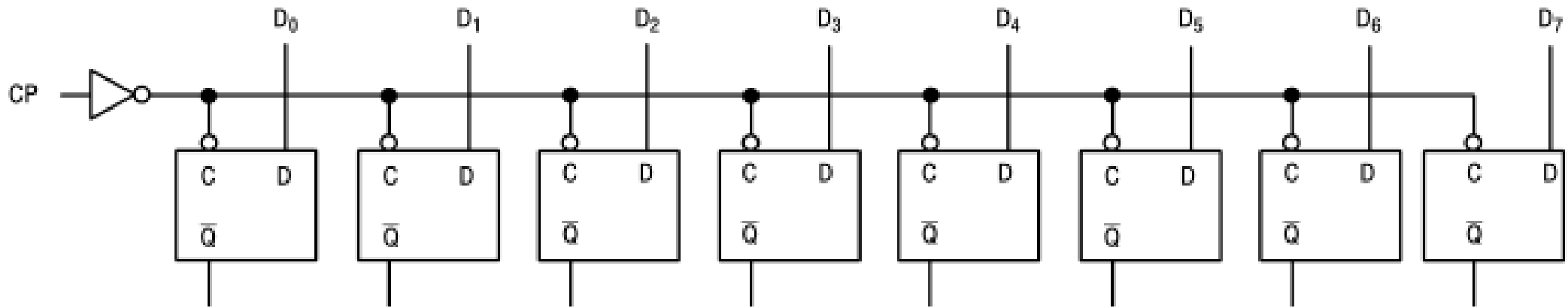
<https://www.relaycomputer.co.uk/>

- <https://web.cecs.pdx.edu/~harry/Relay/RelayPaper.pdf>

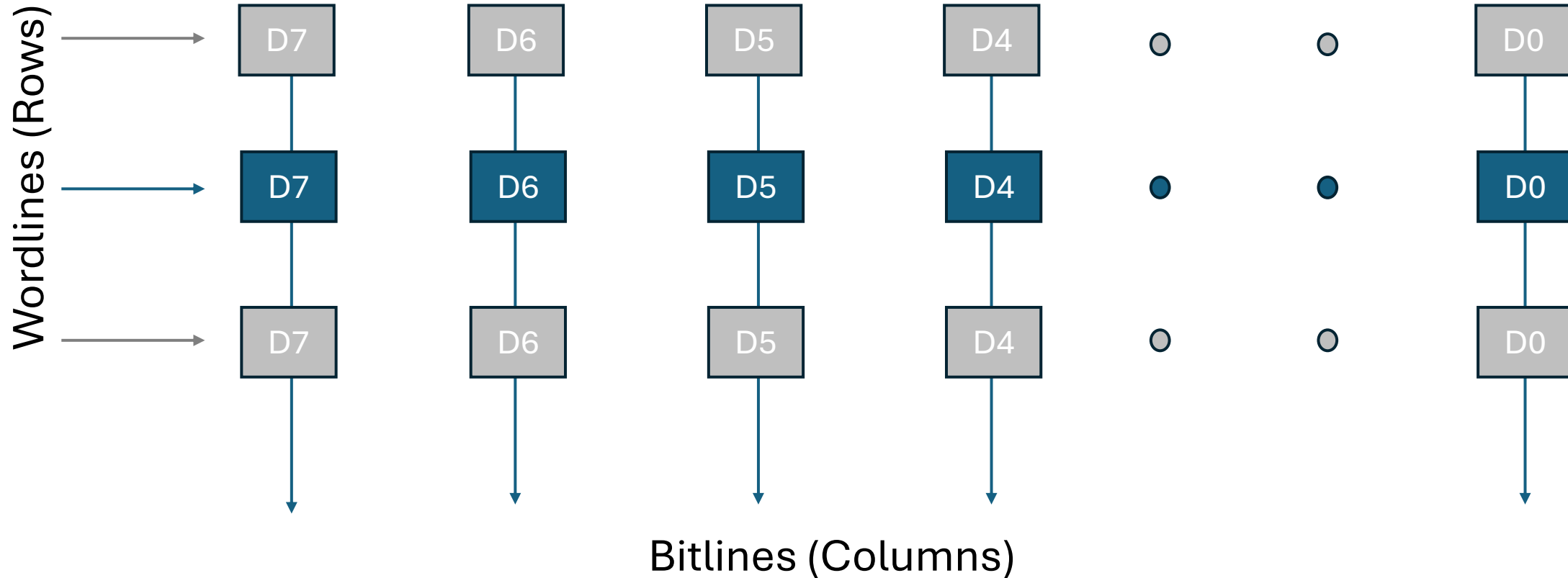
Core Memory Arrays

- The method of arraying core (magnetic) based arrays will follow closely our usage of EEPROM/FLASH Arrays.
- Including ideas of “sensing” a stored charge, and “programming”

Registers



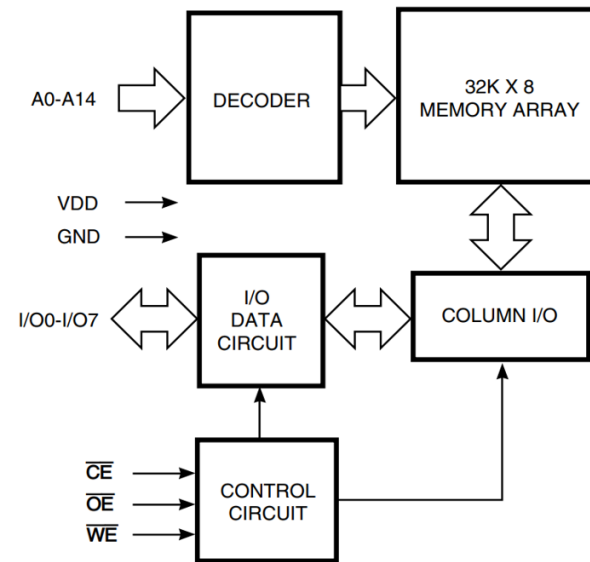
Storage Arrays



Static RAM (SRAM) Access

From Lecture 4.1: Memory Busses

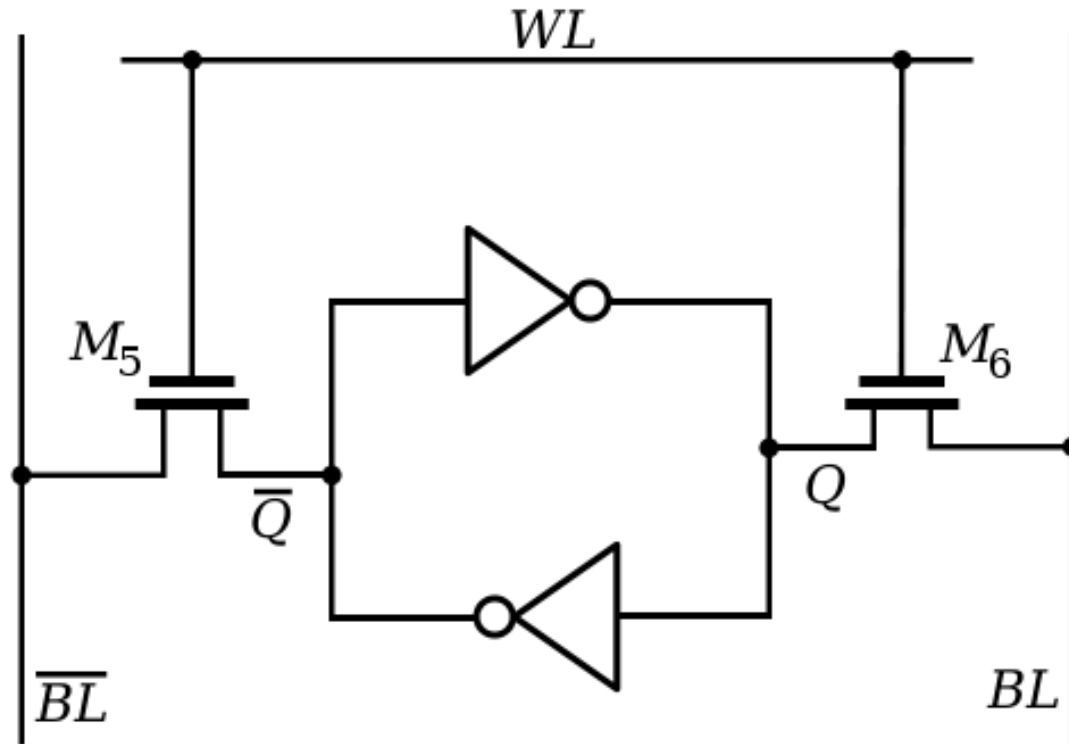
SRAM Configuration



Wordline = Decoder selects word

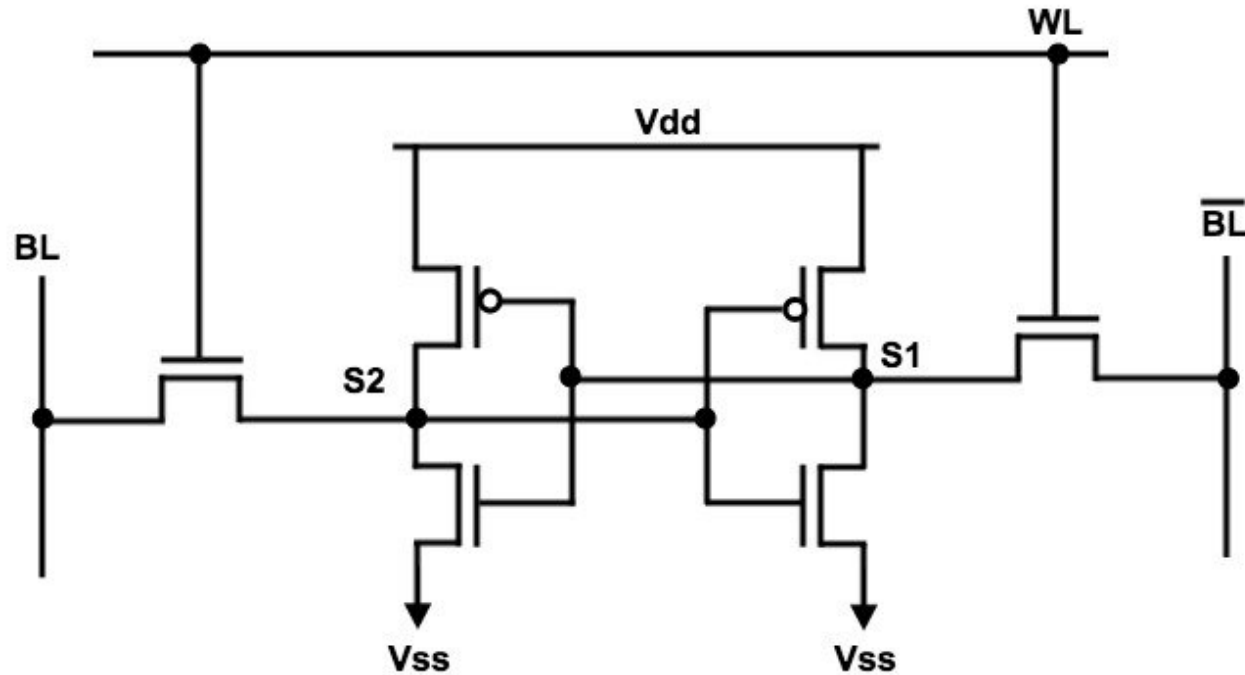
Column I/O = Data Bits

Static RAM (SRAM) Cells



- SRAM cells have two inverters
- Bitlines (read) are read differentially
- A write simply drives a strong signal that overpowers the inverter storage
- A read uses a weak current to detect the voltage

Static RAM (SRAM) Cells



Implementation has 6 transistors

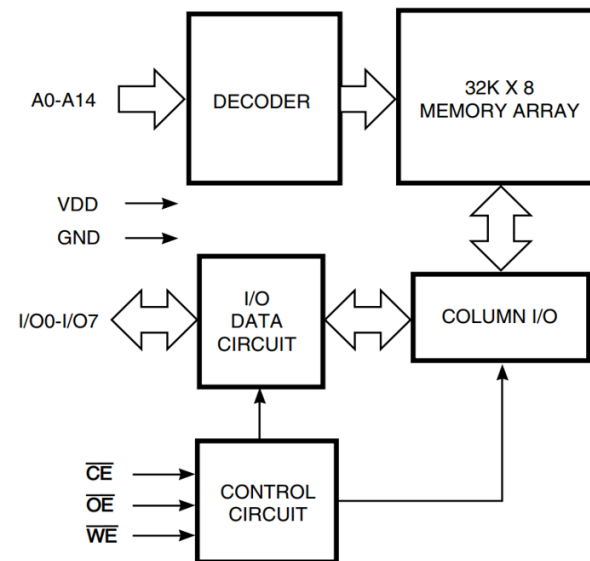
(6 transistors = 1 bit)

More transistors needed in some situations (lower voltages etc)

SRAM Access

From Lecture 4.1: Memory Busses

SRAM Configuration



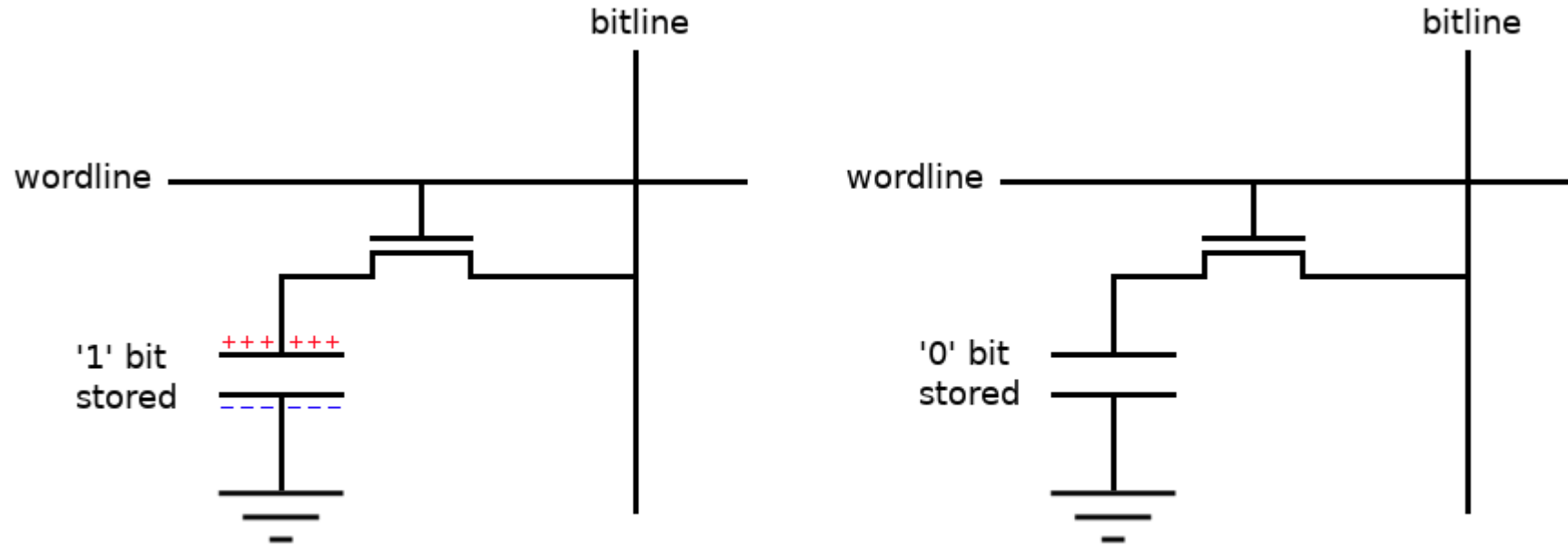
Random Access: simply enable the wordline and the bits are available.

No clock needed!

SRAM Memory Properties

- Equal access time across memory array
- Read & write are similar in access
- Unlimited read/write cycles
- Requires power to maintain data
- Uses 6 transistors per bit (minimum)

Dynamic RAM (DRAM)



DRAM stores the charge on a capacitor (not using a logic element)

DOWNSIDE: Reads are destructive, capacitor will discharge quickly over time (needs a “refresh”)

DRAM Fundamentals

- Dynamic RAM (DRAM) requires a controller that is actively working to refresh rows
 - Typically in range of every $\sim 32 - 64$ mS
- The savings per cell are so high that this extra complexity is worth it above some storage size
- Refresh cycles mean:
 - Impossible to have 100% bus usage
 - Some intelligence in read/buffering/refresh cycles can be needed

Sidenote: PSRAM

- You may run across something called “Pseudo SRAM” (PSRAM)



IS66/67WVS4M8ALL/BLL

GENERAL DESCRIPTION

The IS66/67WVS4M8ALL/BLL are integrated memory device containing 32Mb Pseudo Static Random Access Memory using a self-refresh DRAM array organized as 4M words by 8 bits.

The device supports SPI (Serial Peripheral Interface) & QPI (Quad Peripheral Interface) protocols, Very Low Signal

Tries to combine SRAM “ease of use” with DRAM cost/density

e.g., SRAM vs. PSRAM vs. DDR Memory Costs

- Lets say you wanted to add memory to your design:

Type	Amount	Speed (approx.)	Part Number	Price
SRAM	128 KByte	22 MHz	CY62128ELL-45SXI	\$4
SRAM	4 MByte	20 MHz	AS6C3216A-55TIN	\$43
DRAM	32 MByte	143 MHz	AS4C16M16SA-7TCN	\$7
DRAM	128 MByte	800 MHz	IS43TR16640C-125JBLI	\$9
PSRAM	8 MByte	50 MHz	IS66WVR8M8FALL-104NLI	\$8

NOTE: Many subtleties hidden, giving you “order of magnitude” only

DRAM Memory Properties

- Varied access time
- Read & write are similar in access
- Unlimited read/write cycles
- Requires power AND activity to maintain data
- Uses 1 transistor + 1 capacitor per bit (minimum)

EEPROM / FLASH

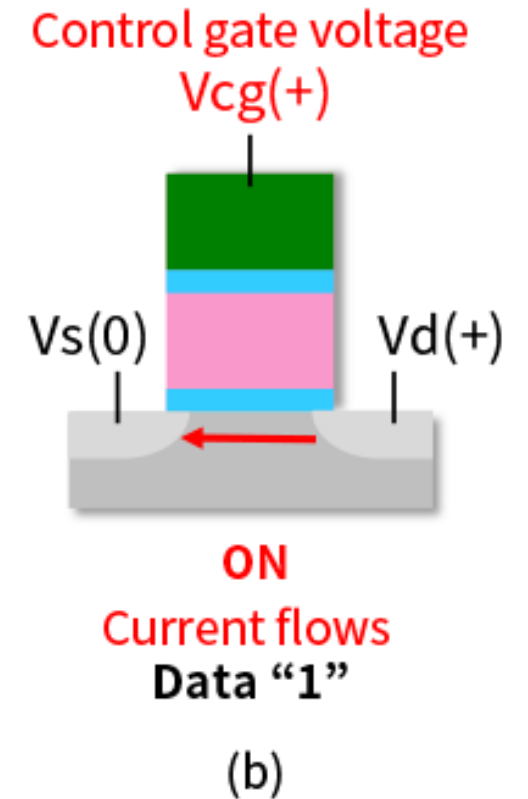
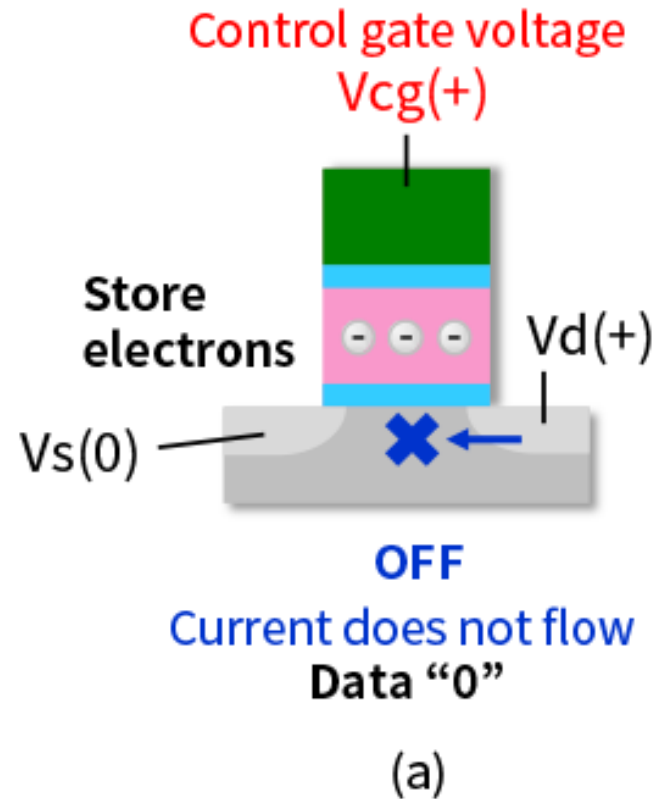
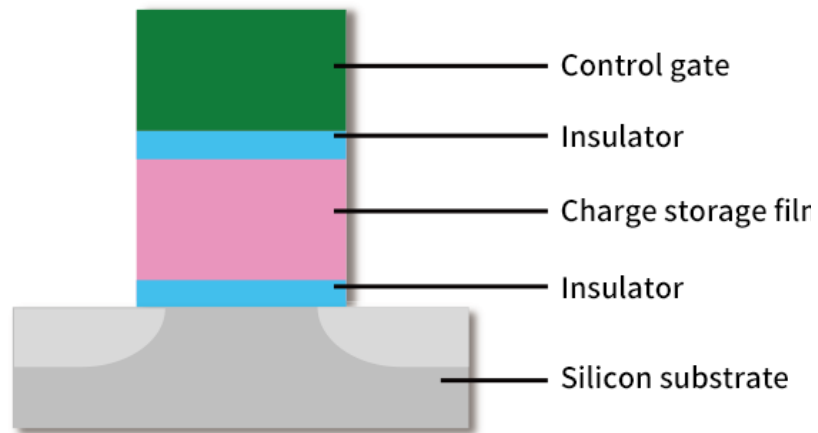
- How do we store data that is non-volatile in computer systems?
- Electronically Erasable Programmable Read Only Memory (EEPROM) & Flash memory are two common technologies you will see
- They both use the same idea – a *floating gate*, but with slightly different implementations

Historical Interests: EPROM

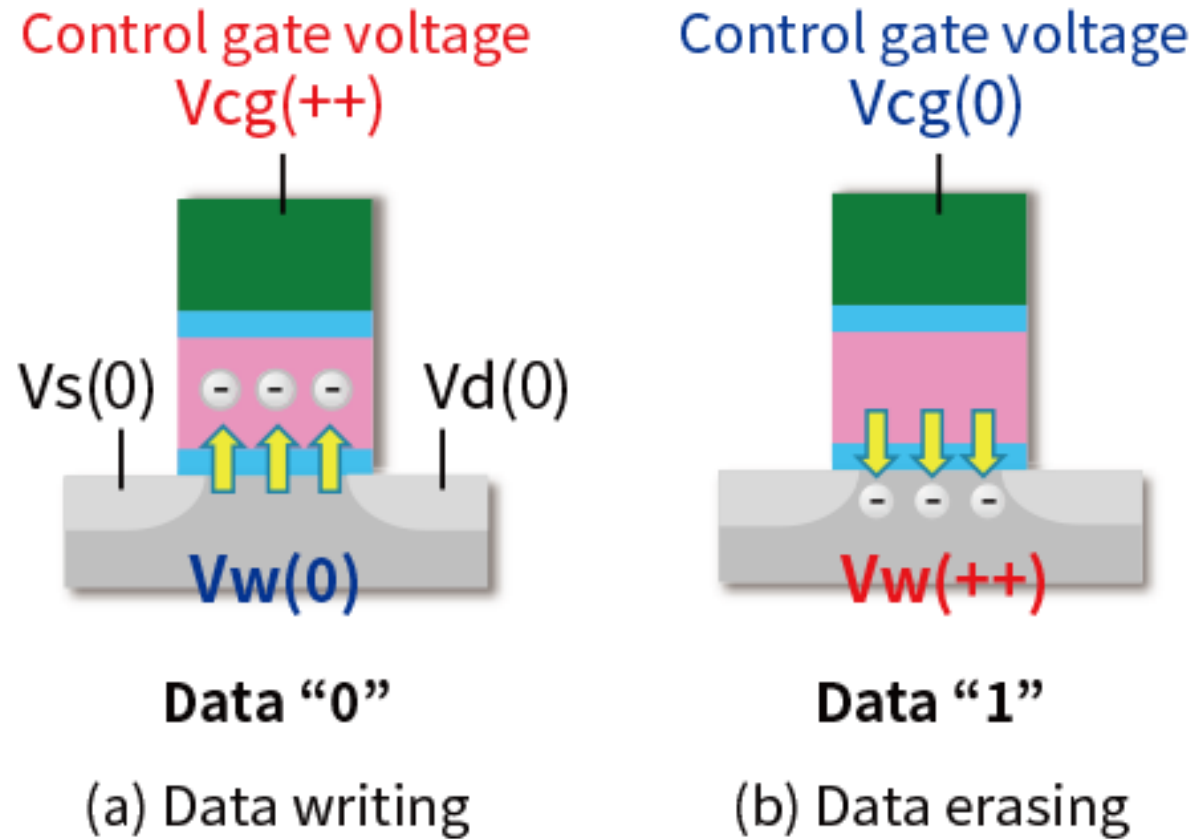
- EPROM = Erasable Programmable Read Only Memory
- Some of our original embedded systems used these – required a long (5-15 min) UV exposure to erase
 - Very annoying for debugging!



Floating Gate Storage - Reads

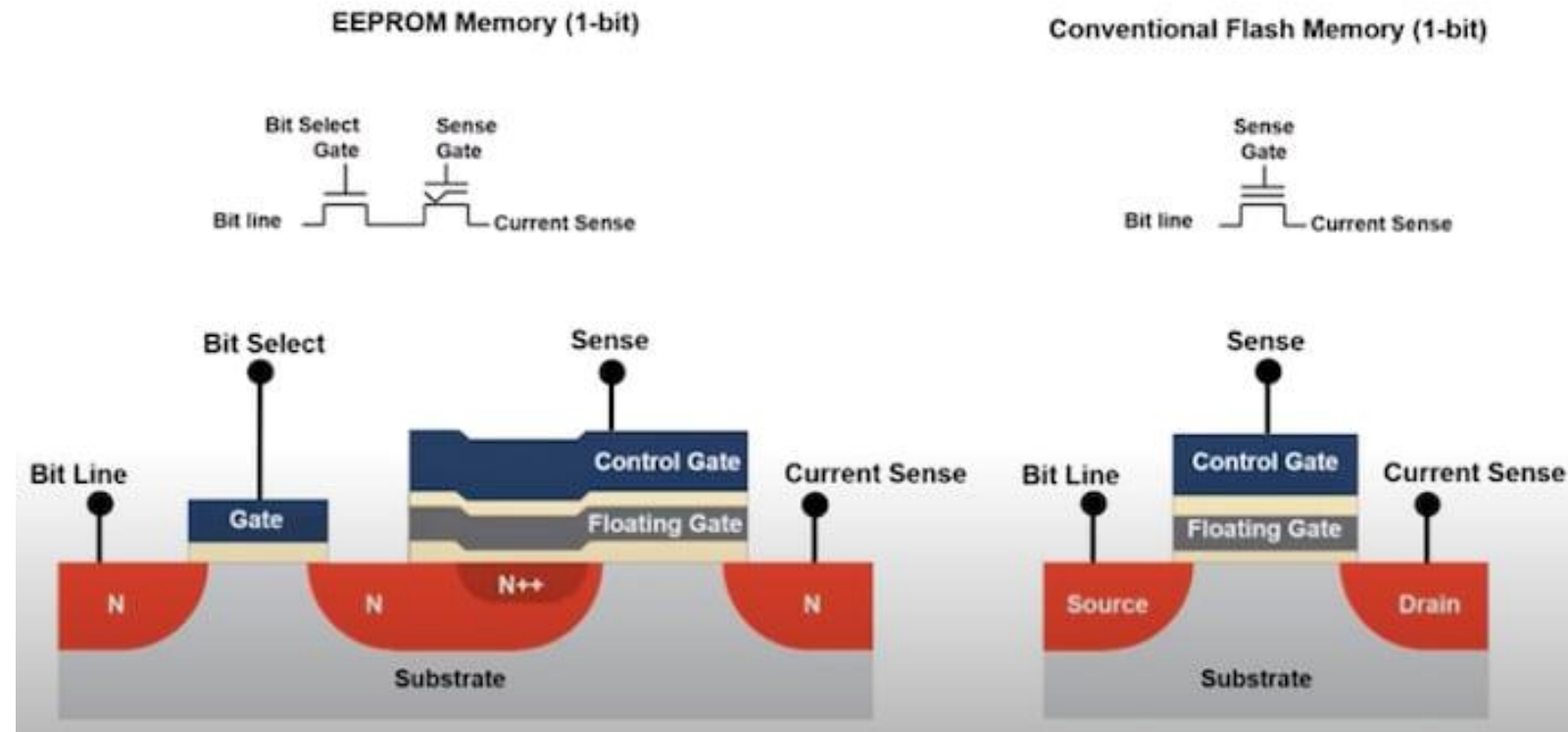


Floating Gate Storage - Writes



EEPROM vs. Flash

- EEPROM has an additional select gate.
- This allows erase/read/write



Read/Program Voltage Notes

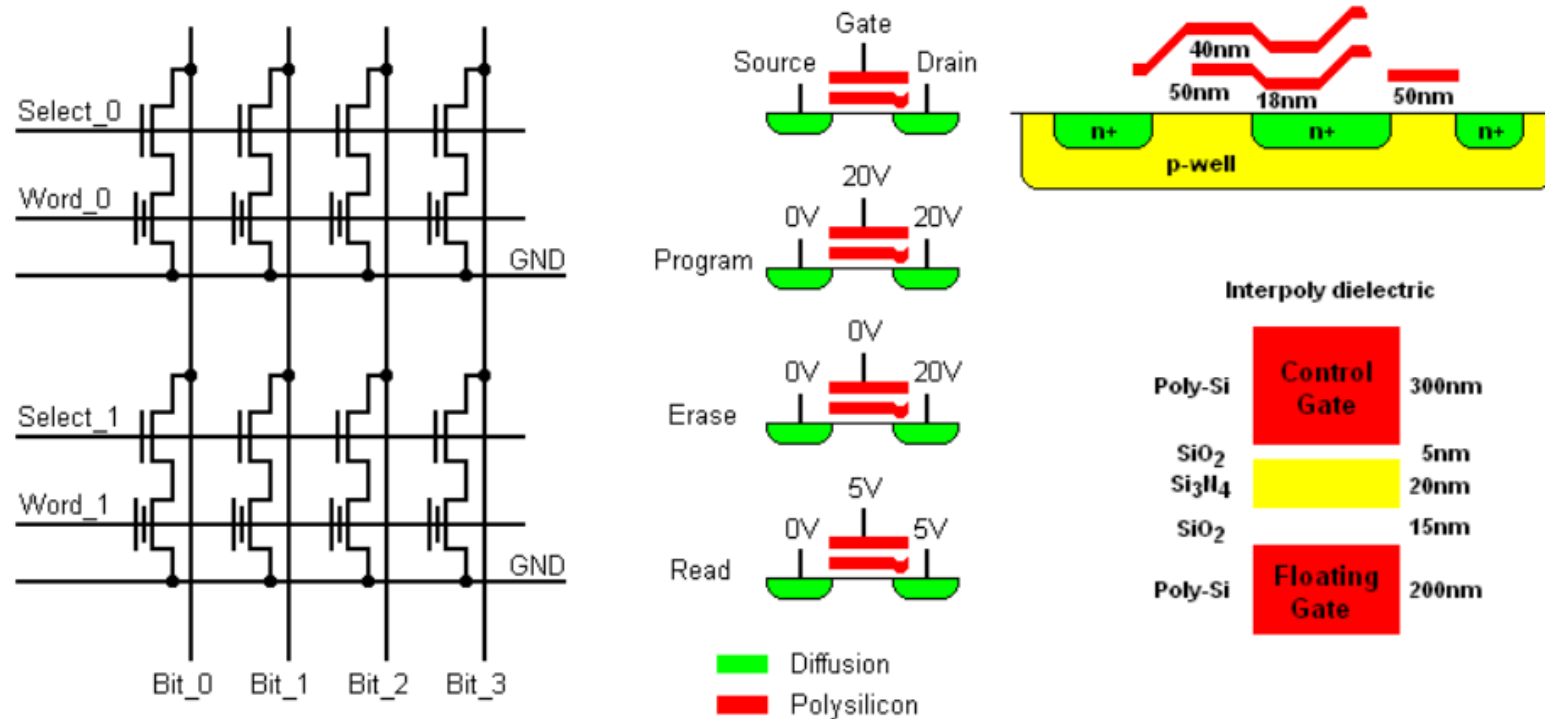


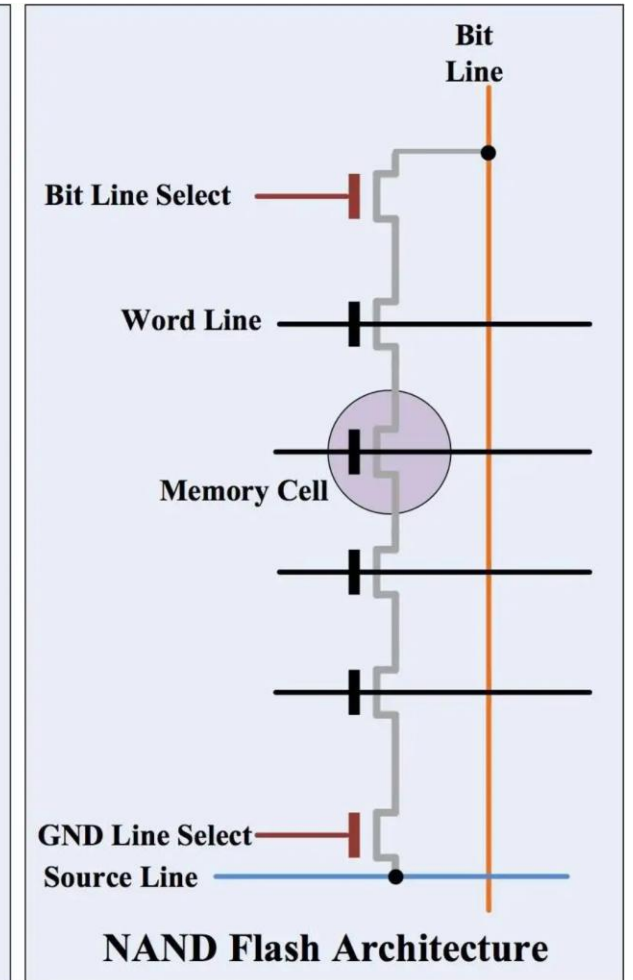
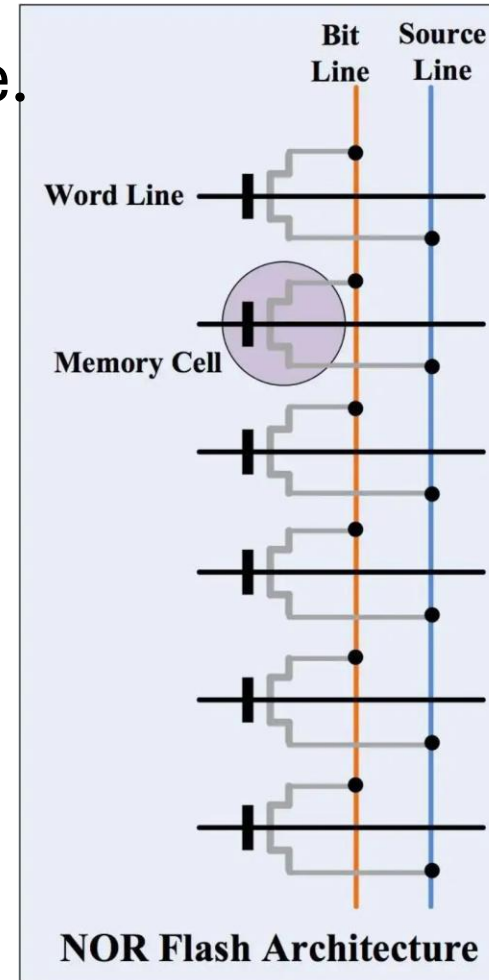
Figure 1: Structure and operation of 2T Flash EEPROM

Flash: NOR vs NAND

- Flash memory saves the select gate.
- Two different organizations:
 - NOR allows per-bit output
 - NAND reduces connection (higher density) but more likely to have errors

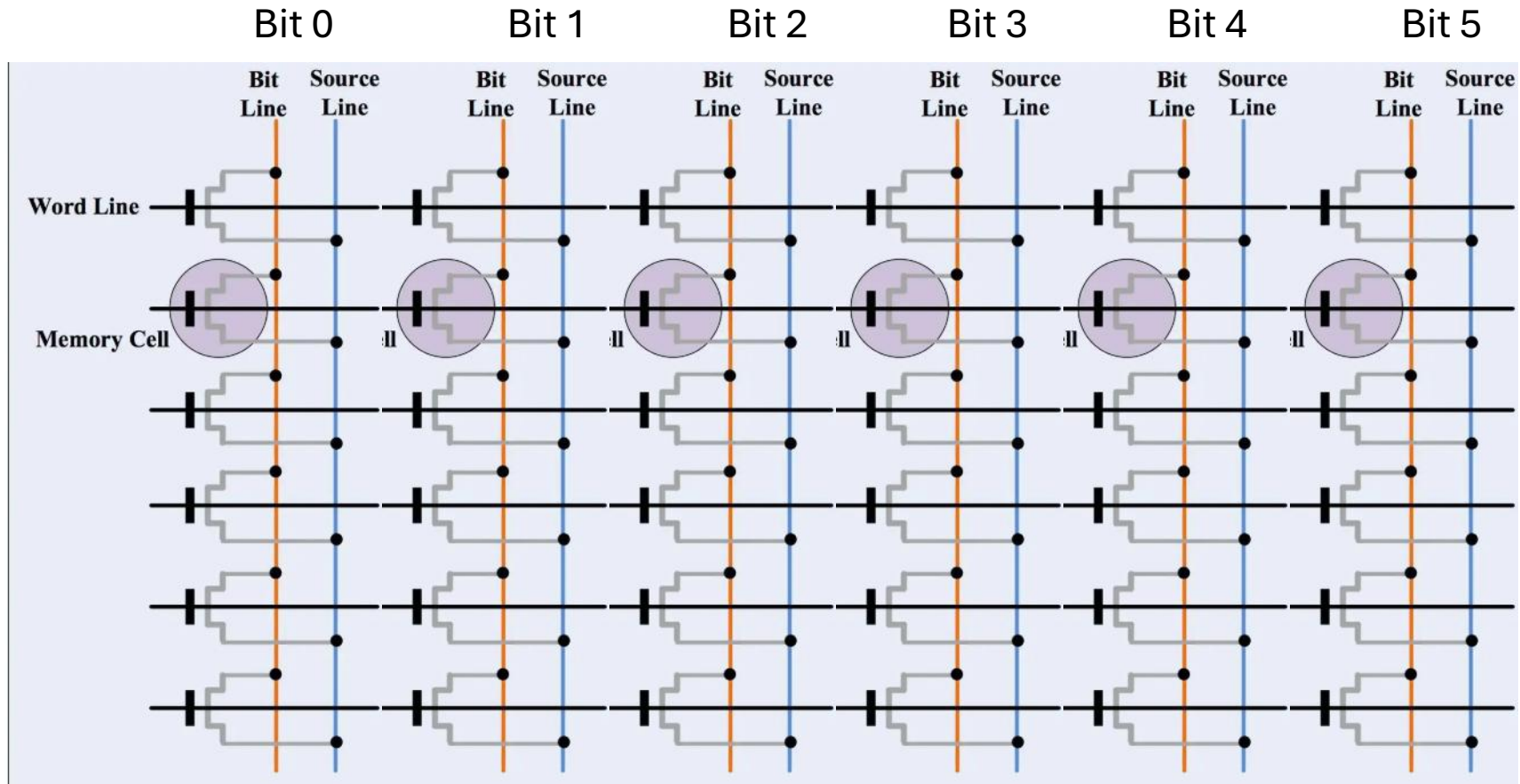
NOR = Microcontroller flash

NAND = SSD, SD Card, etc



<https://www.embedded.com/flash-101-nand-flash-vs-nor-flash/>

e.g., NOR Bitline / Wordline



Flash Memory Pages

- Flash memory normally has pages that define a single “block”:
 - That block is always read & written together
 - Used as it improves density
- To program flash memory:
 - Entire page erased to a “1”
 - Single bits can be programmed to a “0”
- Normally a page is written at once, but you can always program individual bits to “0” later

Flash Memory Writes

- If you needed to change a single byte:
 - Read FLASH page to a buffer
 - Change the required byte
 - Erase entire FLASH page
 - Write entire FLASH page

Flash Memory Organization (Width)

4 Embedded flash memory

4.1 Flash main features

Up to 512 Kbytes of flash memory in STM32F303xD/E, up to 256 Kbytes of flash memory in STM32F303xB/C and STM32F358xC devices and up to 64 Kbytes of flash memory in STM32F303x6/8 and STM32F328x8 devices.

- Memory organization:
 - Main memory block:
128 Kbit x 64 bits in STM32F303xD/E, 64 Kbits x 64 bits in STM32F303xB/C and STM32F358xC devices.
16 Kbit x 64 bits in STM32F303x6/8 and STM32F328x8 devices.
 - Information block:
1280 x 64 bits

Flash memory interface (FLITF) features:

- Read interface with prefetch buffer (2 x 64-bit words)
- Option byte loader
- Flash program/Erase operation
- Read/Write protection
- Low-power mode

In memory with sequential access being common, having extra width is very useful.

Typical in embedded devices (32-bit busses) to see 64-bit or 128-bit (2 or 4 words).

Think e.g., typical instruction flow being linear

Flash “Accelerator”

Instruction fetch

The Cortex[®]-M4 fetches the instruction over the ICode bus and the literal pool (constant/data) over the DCode bus. The prefetch block aims at increasing the efficiency of ICode bus accesses.

Prefetch buffer

The prefetch buffer is 2 blocks wide where each block consists of 8 bytes. The prefetch blocks are direct-mapped. A block can be completely replaced on a single read to the flash memory as the size of the block matches the bandwidth of the flash memory.

The implementation of this prefetch buffer makes a faster CPU execution possible as the CPU fetches one word at a time with the next word readily available in the prefetch buffer. This implies that the acceleration ratio is in the order of 2, assuming that the code is aligned at a 64-bit boundary for the jumps.

Prefetch controller

The prefetch controller decides to access the flash memory depending on the available space in the prefetch buffer. The Controller initiates a read request when there is at least

Flash Timing Demo

- Here is some demo code with 3 examples:
 - Running from SRAM
 - Running from Flash (normal code style)
 - Running from Flash (“jumpy” code that requires more loads)

```
adds r1, r1, r2
eors r2, r2, r1
adds r2, r2, #3
subs r0, r0, #1
bne loop
```

```
b step1
step1: adds ...
b step2
step2: eors ...
b step3
step3: adds ...
subs ...
bne loop
```

Compile project with:

slides_4_3_memoryspeed.c

slides_4_3_loops.s

Compile with “Release” (Optimizations on)

<Live Demo>

measured_flash_jumpy	600016
measured_flash_linear	600016
measured_sram_linear	600018

Slow CPU Speed (Flash faster than CPU)

measured_flash_jumpy	2000016
measured_flash_linear	1300011
measured_sram_linear	600027

High CPU Speed (Flash slower than CPU)

Flash Cycles

- SRAM/DRAM have “unlimited” lifetimes
- Flash/EEPROM have a limited number of cycles
 - Each time the gate charge is erased/written slowly causes additional breakdown
 - Manufacturer can adjust specifications to provide different user requirements. Trade-off between density & reliability/cycles.
- Be aware these specs are often at full temp range (you may see 10-1000x more cycles in practice)
 - E.g., from AtMega328P Datasheet:
 - High endurance non-volatile memory segments
 - 32K bytes of in-system self-programmable flash program memory
 - 1Kbytes EEPROM
 - 2Kbytes internal SRAM
 - Write/erase cycles: 10,000 flash/100,000 EEPROM
 - Optional boot code section with independent lock bits
 - In-system programming by on-chip boot program
 - True read-while-write operation

Wear Leveling

- Cycle life is per cell.
- E.g., if you had a counter that increments every 1 second



Wear Leveling Example

- Writing 1 byte of 256 byte every second:
 - w/o Wear Leveling (same byte)
 - 86 400 cycles per day
 - 2 592 000 cycles per month
 - ~1.1 day lifespan of device @ 100K cycles
 - w/ Wear Leveling (spread among 256 bytes)
 - 337 cycles per day
 - 10 125 cycles per month
 - ~10 month lifespan of device @ 100K cycles

Flash/EEPROM Memory Properties

- Equal access time across memory array
- Read & write are very different
 - Write requires per-page access
- Unlimited read, limited write cycles
- Data is non-volatile (maintained without power)
- Uses 1 transistor + floating gate per bit (minimum)

ROM

- Transistors are either present or absent at interconnections
- Transistor present pulls line LOW
- Used for permanent data defined at manufacture time

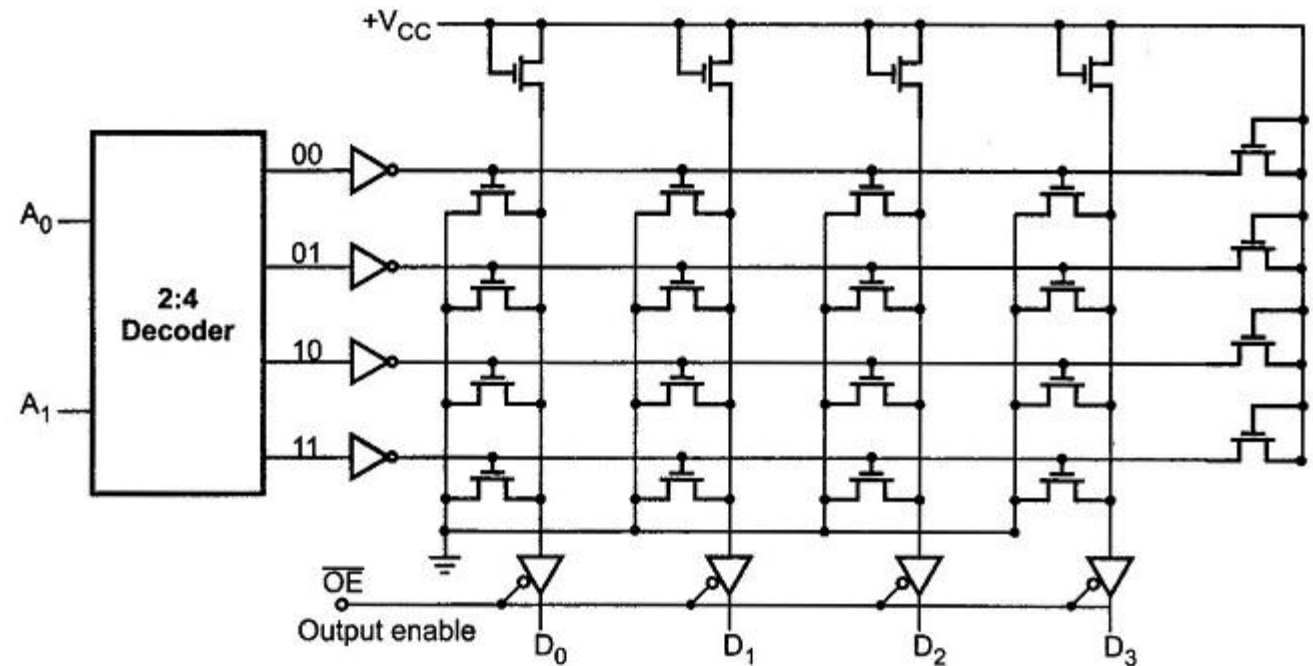
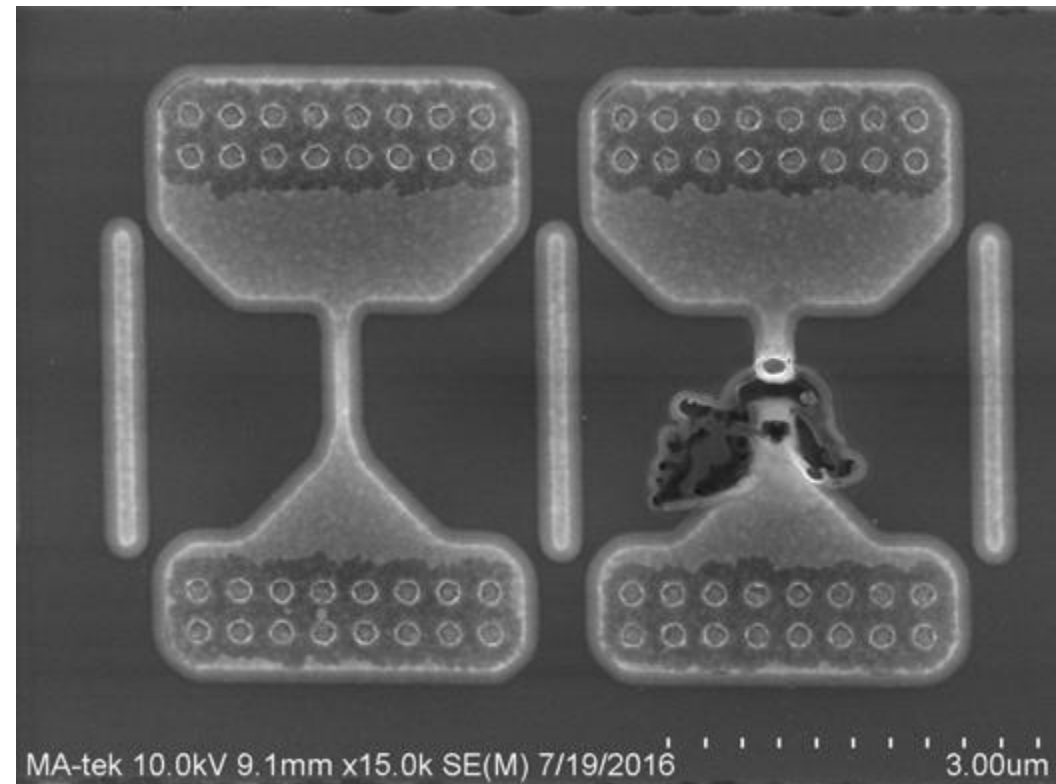


Fig. 3.70 Simple four half-byte ROM

E-Fuses

- Programmable “in the field”.
- Requires higher voltages/current to blow a physical link.
- Often used for security keys and other “cannot change” things
- Different technologies for implementation



Emulated ROM, Flash, and Fuses

- Some devices have only flash memory
- They will attempt to “emulate” the missing bits using control logic:
 - “Fuses” are in Flash

Summary of Memory Types

- Different memory types have different:
 - Speed (access speed)
 - Density
 - Organization
 - Capabilities (read vs write sizes)
 - Storage lifetime
 - Complexity
- Similar organization (wordline vs. bitlines) in most memories
- SRAM and Flash/EEPROM in *all* embedded systems
- DRAM normal in *larger* embedded systems (Linux/Android)